RTL Track

Lab T-RTL2: Multi-Clock Integration

Controlling the Cache from a Second Clock Domain

ASIC Design, Verification and Implementation Onboarding

Kerim Turak

October 18, 2026

Lab objective: Integrate the data cache with a second, unrelated clock domain using the T-RTL1 CDC library, prove with checks and deliberate faults that every crossing works, document every crossing in a CDC report, and time the result with two clocks.

Duration	1.5 weeks (approximately 25–30 hours)
Prerequisites	Lab T-RTL1; Labs R4 and V3 (the cache and its verification environment); Lab P3 (synthesis and timing of the cache)
Designs	`dcache_cdc_top` (new), `dcache` (event port added), CDC library from T-RTL1 (unchanged)
Reused in	The CDC report and the integration pattern are the template for any later multi-clock top

Contents

1 Overview **3**

2 System Specification **3**

2.1 System Structure . . . 3

2.2 Crossings . . . 5

2.3 Ports . . . 5

2.4 Change to the Cache: Event Port . . . 5

2.5 Behavior and Requirements . . . 6

3 Verifying the Top **6**

3.1 Testbench Structure . . . 6

3.2 Checks . . . 7

3.3 Tests and Regression . . . 7

4 Experiment: Breaking the Crossings **8**

5 CDC Crossing Report **9**

5.1 Content . . . 9

5.2 Review Checklist 10
5.3 Cross-Check with STA 10

6 Static Timing with Two Clocks **10**

7 Requirements and Tasks **11**

8 Repository and Flow **12**

9 Deliverables **13**

10 Assessment **13**

11 Review Questions **13**

12 References **14**

1 Overview

A library of correct synchronizers does not make a correct chip. Most CDC bugs found in silicon are *integration* bugs: the right cell used for the wrong kind of signal, two related signals synchronized separately, a protocol assumption of a block violated because the request now arrives from another domain at any time, a reset that is released asynchronously in one domain. They are found by a disciplined crossing review, by testbenches that make the crossings uncertain, and by timing constraints that say exactly what is checked and what is not.

In this lab the cache from Lab R4 runs on the core clock `clk_core`. A configuration and debug block runs on an unrelated, slower clock `clk_cfg`. You build a wrapper `dcache_cdc_top` in `hw/top/dcache_cdc_top/` that lets the configuration domain flush the cache, read a consistent snapshot of its performance counters and receive a trace of every CPU access — using one of each kind of crossing from the T-RTL1 library. Then you break the crossings on purpose and record which check finds each fault, write the CDC crossing report of the top, and run STA with two clocks.

Where this lab sits. It uses the T-RTL1 library unchanged (a bug found in the library is fixed there, with a commit that references the failing test), the cache of Lab R4 with one added output port, the constrained-random environment of Lab V3 for the CPU side, and the synthesis/STA flow of Labs P1–P3. It is the first *top* in the repository (`hw/top/`), as opposed to an IP.

Test term — Constrained-random test. Stimulus drawn at random under constraints that keep it legal (here: CPU requests from the V3 classes, clock periods and phases, cfg request times, trace back-pressure). A pass shows the design works for the situations the seeds produced, not for all; the checks and coverage say which ones were exercised. It is the only practical way to hit the cross-product of CPU state × crossing timing that a multi-clock top has.

By the end of this lab you should be able to:

1. Select the synchronizer for each signal of a real integration and justify the choice.
2. Recognize when a block's protocol precondition ("flush only when the CPU side is idle") must be re-established after a crossing, and design the glue that does it.
3. Verify a multi-clock top end to end: transparency, event counting, snapshot consistency and stream ordering, across clock ratios.
4. Show which CDC faults simulation can find, which need the delay model, and which only a structural review can find.
5. Write a complete CDC crossing report and cross-check it against the inter-clock paths reported by STA.
6. Constrain and time a design with two asynchronous clocks and interpret the result per domain.

2 System Specification

2.1 System Structure

Figure 1 shows what you build. Everything inside the blue frame is the top: the cache with its new event port, the glue (flush gate and flush FSM, snapshot capture, trace write with the drop counter), one T-RTL1 cell per crossing and one `reset_sync` per domain. The memory (`arb_mem` with its loader port) and the CPU master stay in the testbench, exactly as in Lab V3; the cfg block is the testbench's cfg-domain agent.

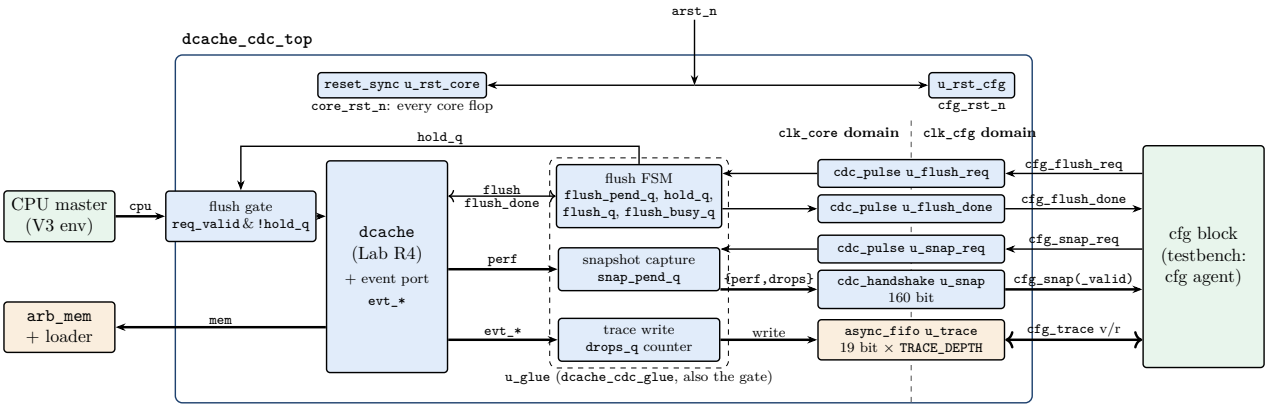


Figure 1: Structure of `dcache_cdc_top`. Blue: blocks you design or reuse (the cache with its new event port, the glue `u_glue` with the flush gate on the CPU request path, the flush FSM, the snapshot capture and the trace write with the drop counter). Each crossing is one T-RTL1 cell, named with its synchronizer type; the dashed line is the clock-domain boundary. One `reset_sync` per domain, both from the raw `arst_n`. Green: testbench.

Figure 2 shows the flush sequence the glue must produce: the request is synchronized, a CPU request already presented is finished, new ones are held off, and only then is the cache flushed.

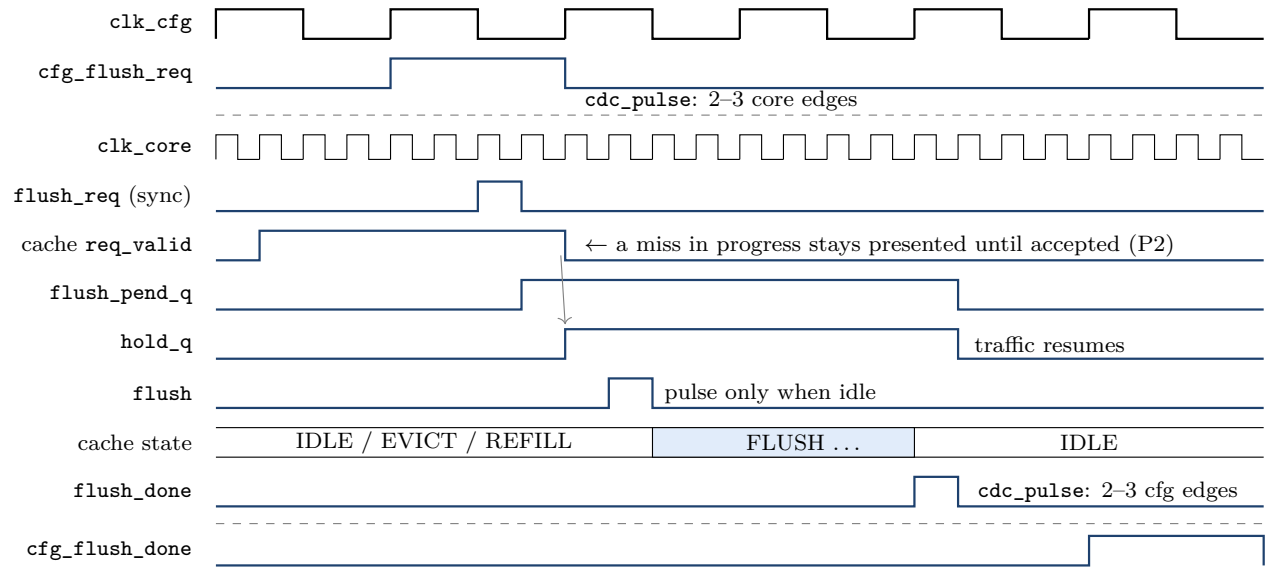


Figure 2: Flush through the glue (core:cfg = 4:1, illustrative cycle counts). The synchronized request sets `flush_pend_q`; the miss already presented to the cache stays presented until it is accepted; only then does the registered `hold_q` block new requests, and one cycle later the cache's `flush` is pulsed with its CPU side idle. The cache's `flush_done` clears the pending state and crosses back as `cfg_flush_done`.

2.2 Crossings

Signal	Direction	Synchronizer	Purpose
Flush request	cfg → core	`cdc_pulse`	Start a cache flush
Flush done	core → cfg	`cdc_pulse`	Report completion
Snapshot request	cfg → core	`cdc_pulse`	Ask for a performance snapshot
Performance snapshot	core → cfg	`cdc_handshake`	Transfer a consistent copy of all four `perf` counters and the trace drop counter
Event trace	core → cfg	`async_fifo`	One entry per accepted CPU request: `{we, result, addr}`
Resets	both	`reset_sync`	One per domain

2.3 Ports

```
module dcache_cdc_top #(
  parameter int ADDR_W      = 10,
  parameter int DW          = 32,
  parameter int LINE_WORDS  = 4,
  parameter int NUM_LINES   = 16,
  parameter int TRACE_DEPTH = 16
) (
  input  logic  arst_n,                // raw asynchronous reset, both domains
  // ---------------- core domain ----------------
  input  logic  clk_core,
  mem_if.slave  cpu,
  mem_if.master mem,
  // ---------------- cfg domain -----------------
  input  logic  clk_cfg,
  input  logic  cfg_flush_req,         // one-cycle pulse
  output logic  cfg_flush_done,        // one-cycle pulse
  input  logic  cfg_snap_req,          // one-cycle pulse, one outstanding
  output logic  cfg_snap_valid,        // one-cycle pulse
  output dcache_cdc_pkg::snap_t  cfg_snap,
  output logic  cfg_trace_valid,
  input  logic  cfg_trace_ready,
  output dcache_cdc_pkg::trace_t cfg_trace
);
```

`snap_t` is a packed struct of the four `perf` counters of Lab R4 and a 32-bit `drops` counter; `trace_t` is `{we, result, addr}` with `result` of type `dcache_pkg::evt_result_e` (`EVT_HIT`, `EVT_MISS_CLEAN`, `EVT_MISS_DIRTY`; the prefix keeps the names distinct from the `HIT`/... of the V3 testbench's `result_e` and of `cache_model_pkg` (Lab T-MOD) — a testbench that imports `dcache_pkg::*` and `cache_model_pkg::*`, as the T-AM labs do, would otherwise see an ambiguous `HIT`). Both types live in `hw/top/dcache_cdc_top/rtl/dcache_cdc_pkg.sv`; the event enum lives in `dcache_pkg`, because the cache produces it.

2.4 Change to the Cache: Event Port

Add an event output to `dcache`: a one-cycle `evt_valid` with `evt_we`, `evt_result` (hit, clean miss, dirty miss) and `evt_addr` for every accepted CPU request (registered, one cycle after the acceptance like a read response, so that the trace logic adds nothing to the CPU request path). The result is the outcome of the *first* lookup of that request: a request that missed, was refilled and then accepted is reported as a miss, not as a hit. The cache must remember this across the miss handling — decide

where.

This is a change to a verified block. Treat it as such: the new port is purely additive, the R4 smoke test and the V3 regression still pass unchanged, and the V3 hit/miss tag model is the reference for a new check that `evt_result` matches the predicted outcome (add it to the V3 scoreboard, not to a new environment).

2.5 Behavior and Requirements

Flush. Lab R4 specifies `flush` as a pulse issued while the CPU side is idle. In the top this precondition no longer holds by itself: the request arrives from another domain, at any time, while the CPU may be in the middle of a miss. The core-domain glue must re-establish it:

1. On the synchronized flush request, set `flush_pending`.
2. While pending, do not present *new* CPU requests to the cache (`cpu.req_ready` stays low towards the CPU). A request that is already presented and not yet accepted must stay presented until accepted — `mem_if` forbids withdrawing a valid request (Lab V2 protocol assertions).
3. When no request is outstanding and no read response is still due, pulse `flush` to the cache.
4. On `flush_done`, clear `flush_pending`, resume CPU traffic and send the done pulse back to the cfg domain.

Explain in `doc/design.md` why this glue sits on the CPU request path, what it does to the timing of that path (you measure it in Section 6), and what happens if a second flush request arrives while one is pending.

Snapshot. On the synchronized snapshot request, if the handshake source is ready, capture `{perf, drops}` of that cycle into `cdc_handshake` and start the transfer. The cfg side may have only one snapshot outstanding; express this as an `assume property` on the cfg port. The four performance counters must be transferred together in one handshake. Explain what could go wrong if each counter had its own handshake (reconvergence).

Trace. Every `evt_valid` writes one entry into the trace FIFO. If the FIFO is full, the event is dropped and the `drops` counter is incremented; the cache never stalls because of the trace. Report the number of drops for each clock ratio.

General. All coding rules of Lab R1 apply; the top is lint-clean; every crossing uses a T-RTL1 library cell, and the top itself contains no synchronizer flip-flop.

3 Verifying the Top

3.1 Testbench Structure

The testbench `hw/top/dcache_cdc_top/dv/tb/tb_top.sv` combines what already exists:

- The Lab V3 environment on the core side: CPU transaction classes, constrained-random tests, flat-memory reference model, tag model, coverage, the memory (`arb_mem`) and its loader. Reuse the files; do not copy them. If the V3 file list mixes the reusable environment with V3's own `tb_top`, split it first (for example `hw/ip/dcache/dv/env/env.f`) and keep V3 passing.
- Two instances of `dv/common/clk_rst_gen.sv` from T-RTL1, with plusargs `+CORE_CLK_NS=`, `+CFG_CLK_NS=`, `+CFG_PHASE_NS=`; random by default.
- A cfg-domain agent (tasks, as in Lab V1) that issues flush and snapshot requests at random times (`+FLUSH_EVERY_NS=`, `+SNAP_EVERY_NS=`) and drains the trace with random back-pressure (`+TRACE_READY_PCT=`).

- Core-domain monitors on the cache's internal `flush`, `flush_done`, `perf` and `evt_*` signals (white-box, by hierarchical reference from a bound checker). The V3 monitors are pointed at the cache's *own* CPU port inside the top (behind the flush gate), the V3 driver at the top's CPU port; that way every V3 check, including the stall-cycle count, still compares the cache with its models.

Test term — Back-pressure test. The consumer refuses data at random (`cfg_trace_ready` low in 100−`TRACE_READY_PCT` % of the cycles), so that the FIFO fills and the full/drop path runs. A pass shows that a slow consumer loses entries only in the specified way (counted drops), never by corruption, duplication or reordering.

On the Synopsys cloud: Simulate this top with VCS: `make sim IP=dcache_cdc_top SIM=vcs` (the cloud default; `WAVES=1` + `make waves` for Verdi). The testbench needs VCS: it reuses the V3 classes and covergroups, and Verilator 5.013 (below the program minimum 5.020) evaluates multi-clock monitors and concurrent assertions with post-edge values (found in Lab T-RTL1). VCS is four-state: an unreset flop of the glue shows up as `X` on `cfg_*`.

3.2 Checks

1. **All Lab V3 checks still pass**: read data, final memory (the final flush is now requested through `cfg_flush_req`) and hit/miss, plus the new event-result check.
2. **Flush:** every flush request in the cfg domain produces exactly one flush in the core domain and exactly one flush-done pulse back; the cache's `flush` is asserted only while the CPU side is idle (an assertion in the core domain).
3. **Snapshot consistency:** every snapshot equals the counter values at one specific core cycle. Record `{perf, drops}` of every core cycle in an associative array indexed by the packed value, with the cycle number as entry; check that the snapshot is in it, and that the cycle lies between the request and the delivery of that snapshot.
4. **Trace:** the event trace read in the cfg domain is the sequence of accepted CPU requests observed in the core domain with some entries missing; the number missing equals the `drops` value of the final snapshot, and no entry is duplicated or reordered.

3.3 Tests and Regression

Test	Purpose
`v3_random`	The V3 random test through the top, one final flush, no cfg traffic otherwise
`flush_storm`	Flush requests at the minimum legal spacing during random CPU traffic, including during misses and dirty evictions
`snap_storm`	Back-to-back snapshots during random traffic; every snapshot checked
`trace_overflow`	Slow cfg clock and low `TRACE_READY_PCT`: many drops, all counted
`trace_lossless`	Fast cfg clock and `TRACE_READY_PCT=100`: show the ratio at which no event is dropped
`mixed`	All of the above at random

What each test is meant to show: `v3_random` — the top is transparent to the CPU (all V3 checks through the glue); `flush_storm` — no flush is lost or doubled and the cache is flushed only when idle, even when requests land during misses and evictions; `snap_storm` — every snapshot is one core cycle's counters; `trace_overflow` — drops happen and every one is counted; `trace_lossless` — the ratio and depth at which nothing is dropped; `mixed` — the interactions. Give the clock ratio in the test name (for example `flush_storm_4to1`, `_10to1`, `_1to3`, `_near`, `_rand`) so that every regression run has its own log, as in T-RTL1.

Test term — Stress test. The storm tests run one interface at the highest rate its protocol allows (a new flush or snapshot right after the previous one completed) while the CPU side runs random traffic. A pass shows that no request is lost or duplicated at the maximum rate; it is where back-to-back corner cases (a request in the cycle a handshake completes, a flush during a dirty miss) are hit.

Test term — Regression. All tests × clock ratios × seeds × the two FIFO depths, run by one command (`make regress`) with a PASS/FAIL table. A pass is the evidence the CDC report cites; it is rerun after every change, so that a fix in one place does not silently break another ratio.

The regression runs every test with `CDC_RANDOM_DELAY`, 5 seeds, at the clock ratios core:cfg ≈ 4:1, 10:1, 1:3, nearly equal (10.0 and 10.1 ns) and random. Record the drop counts per ratio for `TRACE_DEPTH` 4 and 16 and explain them from the ratio and the access rate.

Measure and report: the flush round-trip latency (cfg cycles from request to done) and the snapshot latency, each for three ratios, and compare them with what the T-RTL1 latency measurements predict.

4 Experiment: Breaking the Crossings

For each fault below, run the regression *with and without* `CDC_RANDOM_DELAY` and record which check, if any, detects it.

Test term — Error-injection (negative) test. A known bug is inserted on purpose and the verification is expected to *fail*. A failing run proves that the checks can see this kind of bug; a fault that passes everywhere shows a blind spot of the method (here: simulation, the delay model, or the structural CDC check), which is the result you record.

Make each fault on a separate commit of an experiment branch (`exp/trtl2-cdc-faults`) that is never merged; record the commit hashes in the results table. If a fault requires changing a library block, change a copy in the experiment branch, never the library on the lab branch.

Fault	Question to answer
FIFO pointers synchronized in binary instead of Gray	Does RTL simulation without the delay model find it? Which check finds it with the model?
Flush request crossed with a plain `cdc_sync` instead of `cdc_pulse`, fast cfg clock	How many flushes are lost, and at which clock ratios?
Performance counters crossed with `cdc_sync` instead of the handshake	Does the snapshot consistency check catch the incoherent values?
Combinational logic between a source flip-flop and `cdc_sync`	Can any simulation find it? What would find it in practice?
Flush glue removed (raw synchronized pulse to the cache)	Which check or assertion fires, and how often in each ratio?
Optional: one handshake per counter	Show the reconvergence failure with the consistency check

Results table format, in `doc/verification.md`:

Fault (commit)	No delay model	Delay model	Detected by / explanation
binary pointers (`a1b2c3d`)	pass / fail	pass / fail	check, seed, ratio, first failing time
...			

The fourth fault is the important lesson: some CDC bugs cannot be found by simulation at all, only by structural analysis and design review — the CDC crossing report of the next section. Run the structural check on every fault as well (`make cdc`, next section) and record which rule fires; a fault the CDC tool does not see (a correct synchronizer used for the wrong *kind* of signal, a removed glue) is a case for simulation.

5 CDC Crossing Report

The crossing report is the document a CDC sign-off review is held on. In a commercial flow a CDC analysis tool generates most of it and the engineer waives or fixes each finding; here you write it by hand and cross-check it with STA. It goes into `hw/top/dcache_cdc_top/doc/design.md`.

On the Synopsys cloud: SpyGlass CDC generates the crossing inventory on the cloud: `make cdc IP=dcache_cdc_top` (`flow/mk/cdc.mk`, T-RTL1). It needs a top without interface ports (a small wrapper, as for lint) and an SGDC file (`syn/<top>.sgdc`): `clock -name clk_core -domain core -period 10 -edge {0 5}`, the same for `clk_cfg`, `reset -name arst_n -async`, `abstract_port` for every input (its clock), and `fifo -memory ... -rd_ptr ... -wr_ptr` for the trace FIFO. Goals: `cdc/cdc_setup_check` (are clocks, resets and port domains complete?), `cdc/cdc_verify_struct` (structure: synchronizers, convergence, glitch-prone logic) and, with `CDC_GOALS="... cdc/cdc_verify"`, the formal checks (Gray encoding, data hold). How to read `consolidated_reports/<top>_cdc_<goal>/moresimple.rpt`: `Ac_sync01/02` (Info) are the synchronized crossings — one per row of your report; `Ac_unsync01/02` are unsynchronized crossings (always a bug); `Ac_conv01/02/04` are convergence of separately synchronized signals (review each: fix or waive with the reason, the message says whether the formal Gray check `PASSED`/`FAILED`/was `DISABLED`); `Ac_glitch03` is combinational logic in front of a synchronizer. Waivers go to `lint/<ip>_cdc.awl`, each with a comment; the correct top should end with 0 errors and 0 warnings (`CDC_FAIL_ON=warning`). VC SpyGlass CDC (`vc_static_shell`) is the alternative tool.

5.1 Content

One row per crossing *signal or bus* of the top, including the crossings inside the library instances (copy the T-RTL1 crossing tables with the instance path):

Signal / instance	From	To	Synchronizer	Why safe; evidence
`u_flush_req` toggle	cfg	core	`cdc_pulse`	Pulse spacing assumed (assume property ...); check 2; regression ...
`u_snap` data	core	cfg	`cdc_handshake`	Data held while `req`; assertion ...; check 3
`u_trace wptr_gray`	core	cfg	`cdc_sync` in `async_fifo`	Gray, registered; max delay ...
`arst_n` release	async	core, cfg	`reset_sync` ×2	Async assert, sync release per domain
...				

Below the table, list every *protocol assumption* a crossing relies on (pulse spacing, one outstanding snapshot, data held during the handshake) and where it is checked.

5.2 Review Checklist

Answer each item for the top, with a reference to the code:

1. Every signal entering a synchronizer comes directly from a flip-flop of the source domain.
2. No logic between synchronizer stages; first stage drives only the second.
3. No signal of one domain is used in the other except through a row of the table.
4. No reconvergence: no two separately synchronized signals are combined in the destination domain.
5. Every multi-bit value crosses through Gray code, a handshake or the FIFO.
6. Every domain has its own reset synchronizer; no flip-flop uses the raw `arst_n`.
7. Every protocol assumption is an `assume property` or an assertion, and it was active in the regression.

5.3 Cross-Check with STA

Before adding the clock groups, STA reports every path that starts in one clock domain and ends in the other. That list is an independent, tool-generated inventory of the crossings:

```
report_checks -from [get_clocks clk_core] -to [get_clocks clk_cfg] \
              -group_count 200 -endpoint_count 1 -format end
report_checks -from [get_clocks clk_cfg]  -to [get_clocks clk_core] \
              -group_count 200 -endpoint_count 1 -format end
```

(Older OpenSTA versions call `-group_count` `-group_path_count`.)

On the Synopsys cloud: PrimeTime (`STA_TOOL=pt`): the inter-clock paths are `report_timing -from [get_clocks clk_core] -to [get_clocks clk_cfg] -max_paths 1000 -nworst 1 -slack_lesser_than 1e9` (PrimeTime's `-max_paths` alone implies `-slack_lesser_than 0`, i.e. only violators), or the same in a Tcl loop with `get_timing_paths` to print one line per endpoint and map it to a report row. The clock-pair summary is `check_timing -include {clock_crossing} -verbose`: it lists the interacting clocks and marks a pair whose paths are all false (`*`) or partly false (`#`). (Vivado's `report_clock_interaction` does not exist in PrimeTime; `report_clock_crossing` exists in PrimeTime 2025.06 but printed nothing in our setup.) Synthesize with `SYN_FLATTEN=0` so that the endpoint names keep the instance path (`u_trace/u_sync_w2r/sync_q_reg_0__3_`).

Every endpoint must be a first synchronizer stage (`*sync_q[0]*`), the FIFO read data path, or the handshake capture register; every endpoint must map to a row of the report. An endpoint that maps to no row is an unsynchronized crossing — a bug, whatever simulation says. Keep the CDC library instances unflattened in synthesis so that the instance names survive (the hierarchy-aware synthesis of Lab P3).

6 Static Timing with Two Clocks

```
# hw/top/dcache_cdc_top/syn/dcache_cdc_top.sdc
create_clock -name clk_core -period 10.0 [get_ports clk_core]
create_clock -name clk_cfg  -period 40.0 [get_ports clk_cfg]
set_clock_groups -asynchronous -group {clk_core} -group {clk_cfg}
```

```
# I/O: cpu_* and mem_* relative to clk_core, cfg_* relative to clk_cfg,
# delays and loads as in the dcache SDC of Lab P3
```

On the Synopsys cloud: `set_clock_groups -asynchronous` versus `set_max_delay`. The clock group tells the tool the two clocks have no phase relation and removes *every* check between them — correct for a synchronizer input (its timing is not deterministic), but it also hides a crossing that has no synchronizer and lets the Gray-pointer bits arrive any number of cycles apart. A `set_max_delay` of one destination period on the Gray-pointer paths (PrimeTime: `-from` the pointer register `-to` the first synchronizer stage; add false paths only `-to` the other synchronizer stages, because in PrimeTime a false path wins over a max delay) keeps those paths checked. On the cloud: Fusion Compiler (`SYN_TOOL=fc`) for synthesis, PrimeTime (`STA_TOOL=pt`) for timing; PrimeTime reports recovery/removal in the path group `**async_default**`. The bring-up library `class.db` has no recovery/removal arcs, so that report stays empty with it.

Tasks:

1. Synthesize the top for sky130 with the CDC instances kept as hierarchy. Report the area of the CDC logic (library instances plus glue) relative to the cache.
2. Run STA without the clock group and produce the inter-clock path list of Section 5; reconcile it with the report.
3. Run STA with the clock group. Report WNS and the critical path per clock. Find the minimum `clk_core` period and compare it with the `dcache` result of Lab P3: did the flush glue on the CPU request path lengthen the critical path? If yes, fix it (for example by registering the gating decision) and explain what the fix costs.
4. Add the max-delay constraint on the trace FIFO's Gray pointers (as in T-RTL1) and show that those paths are now checked.
5. Check the recovery/removal checks of every flip-flop: all of them must be relative to the clock of their own domain, through the reset synchronizer.

7 Requirements and Tasks

1. Add the event port to `dcache` and the event-result check to the V3 scoreboard; `make regress IP=dcache` still passes.
2. Write `doc/spec.md` of the top (ports, crossings, protocol of each cfg-side signal, assumptions).
3. Implement `dcache_cdc_pkg`, the glue and `dcache_cdc_top`, lint-clean, using only T-RTL1 cells for crossings.
4. Build the testbench and the four checks, the assertions (flush only when idle, one outstanding snapshot, cfg pulse spacing) and cover properties (flush during a dirty miss; snapshot during a refill; trace FIFO full; drop counted).
5. Write `dv/testplan.md` and `dv/regress.list`; run the regression over all tests, ratios and 5 seeds with `CDC_RANDOM_DELAY`.
6. Measure flush and snapshot latencies and trace drop counts per ratio and FIFO depth.
7. Run the fault experiment and fill the results table.
8. Write the CDC crossing report and the review checklist answers; reconcile with the STA inter-clock path list.
9. Run the two-clock STA tasks; record the results in `doc/ppa.md`.
10. `make check-all` still passes.

8 Repository and Flow

```
hw/top/dcache_cdc_top/
+-- README.md
+-- Makefile
+-- doc/        spec.md, design.md (incl. CDC crossing report),
|               verification.md, ppa.md
+-- rtl/        dcache_cdc_pkg.sv, dcache_cdc_glue.sv,
|               dcache_cdc_top.sv, dcache_cdc_top.f
+-- lint/       dcache_cdc_top.vlt, wrapper with plain ports (lint, CDC),
|               SpyGlass waivers (.awl, _cdc.awl)
+-- dv/
|   +-- testplan.md
|   +-- tb/     tb_top.sv
|   +-- env/    cfg agent, snapshot/trace/flush checkers
|   +-- tests/  test selection (+TEST=)
|   +-- sva/    dcache_cdc_top_sva.sv, dcache_cdc_top_bind.sv
|   +-- regress.list
|   +-- dv.f
+-- syn/        dcache_cdc_top.sdc (+ _noasync, _maxdelay), SGDC for make cdc
+-- exp/        the faults of the experiment as patches + a script that runs them
hw/ip/dcache/rtl/       dcache.sv, dcache_pkg.sv: event port (changed)
hw/ip/dcache/dv/        event-result check; env file list split if needed
```

The RTL file list reuses the other blocks with nested `-f`, never copies:

```
// hw/top/dcache_cdc_top/rtl/dcache_cdc_top.f
-f ${REPO_ROOT}/hw/ip/cdc/rtl/cdc.f
-f ${REPO_ROOT}/hw/ip/dcache/rtl/dcache.f
+incdir+${REPO_ROOT}/hw/top/dcache_cdc_top/rtl
${REPO_ROOT}/hw/top/dcache_cdc_top/rtl/dcache_cdc_pkg.sv
${REPO_ROOT}/hw/top/dcache_cdc_top/rtl/dcache_cdc_glue.sv
${REPO_ROOT}/hw/top/dcache_cdc_top/rtl/dcache_cdc_top.sv
```

The Makefile has the same structure as an IP Makefile (`IP := dcache_cdc_top`, `DUT_TOP`, `TB_TOP := tb_top`, `SDC`, `PARAMS`) and includes the same `flow/mk/*.mk`. The top-level Makefile delegates `IP=<name>` to `hw/ip/<name>/`; extend it so that it also finds `hw/top/<name>/`. Output goes to `build/dcache_cdc_top/<cfg>/`.

```
make lint    IP=dcache_cdc_top
make sim     IP=dcache_cdc_top TEST=flush_storm DEFINES=CDC_RANDOM_DELAY \
             PLUSARGS="+CORE_CLK_NS=10.0 +CFG_CLK_NS=40.0" SEED=3
make sim     IP=dcache_cdc_top TEST=trace_overflow PARAMS="TRACE_DEPTH=4" \
             DEFINES=CDC_RANDOM_DELAY
make regress IP=dcache_cdc_top
make cdc     IP=dcache_cdc_top          # SpyGlass CDC (structural)
make synth   IP=dcache_cdc_top
make sta     IP=dcache_cdc_top
make regress IP=dcache          # V3 still passes with the event port
```

Branch `lab/trtl2-dcache-cdc`, tag `trtl2-done`; the experiment branch `exp/trtl2-cdc-faults` is pushed but not merged.

9 Deliverables

The merge request contains `hw/top/dcache_cdc_top/`, the `dcache` event-port change with its V3 check, and the top-level Makefile change.

- `README.md`: what the top does, how to run every target, results summary, known limitations.
- `doc/spec.md`: ports, crossings, cfg-side protocol and assumptions.
- `doc/design.md`: block diagram, flush glue design and its timing impact, the snapshot and trace design, the **CDC crossing report** with protocol assumptions, the review checklist answers and the STA reconciliation.
- `doc/verification.md`: testplan status, regression summary (all tests × ratios × 5 seeds with the delay model), snapshot/trace/flush check results, latency and drop tables, the fault experiment table with commit hashes.
- `doc/ppa.md`: area of the CDC logic vs the cache, STA per clock with and without clock groups, core Fmax compared with Lab P3, the max-delay result.

10 Assessment

Criterion	**Weight**
Integration RTL: top, glue, event port, correct use of the library	25%
Testbench and the four checks; regression across clock ratios	25%
Fault experiment and its analysis	15%
CDC crossing report, checklist and STA reconciliation	15%
Two-clock STA and timing impact of the glue	10%
Documentation and reproducibility	10%
Total	100%

11 Review Questions

1. Why is the flush request crossed with `cdc_pulse` and not with `cdc_sync`? Under which clock ratio would `cdc_sync` happen to work?
2. Which R4 precondition did the crossing break, and how does your glue re-establish it without violating the `mem_if` protocol?
3. Why must all counters travel in one handshake? Describe a concrete wrong snapshot that separate handshakes could produce.
4. Why is a dropped trace event acceptable while a lost flush request is not? How did you size `TRACE_DEPTH`?
5. Which fault from the experiment could no simulation find, and which part of your flow finds it?
6. Why did the binary-pointer fault pass without the delay model?
7. Which STA endpoints are legitimate crossings, and how do you know that the list is complete?
8. What does the clock-group constraint hide, and how does the max-delay constraint recover part of it?

9. Did the CDC wrapper change the core Fmax? Explain from the critical path.
10. A new engineer adds a status bit from the core to the cfg domain. What must they add to the code, the report and the testbench?

12 References

- Clifford E. Cummings, Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog, SNUG 2008 (reconvergence, multi-bit crossings, CDC verification).
- Clifford E. Cummings, Simulation and Synthesis Techniques for Asynchronous FIFO Design, SNUG 2002.
- Lab T-RTL1 of this program: the CDC library, its crossing tables and its SDC.
- OpenTitan: `hw/top_earlgrey` clock and reset structure and the CDC methodology notes in its documentation, as an example of a multi-clock SoC.
- OpenSTA documentation: `report_checks` options, `set_clock_groups`, `set_max_delay`.